

RBC_sim_example

May 18, 2024

This notebook presents guides for simulating 2D horizontally-periodic Rayleigh-Benard convection using Dedalus framework as written in following [Dedalus website](#)

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import dedalus.public as d3
```

Computational domain of problem is $[L_x \times L_z] := [4 \times 1]$ with a corresponding mesh of $[N_x \times N_z] := [256 \times 64]$

```
[ ]: # Parameters
Xmin, Xmax = 0., 1.      # computational domain of x-dimension
Zmin, Zmax = 0., 1.      # computational domain of z-dimension
Nx, Nz = 128, 128        # number of points
```

The system is governed by

$$\nabla \cdot \vec{u} = 0, \quad (1)$$

$$\partial_t \vec{u} + \vec{u} \cdot \nabla \vec{u} = \nu \nabla^2 \vec{u} - \nabla p + B \vec{e}_z, \quad (2)$$

$$\partial_t B + \vec{u} \cdot \nabla B = \kappa \nabla^2 B. \quad (3)$$

where $\vec{u}$, p , B , $\vec{e}_z$, κ , and ν are the fluid velocity, pressure, buoyancy term, unit vector in the vertical, thermal diffusivity, and kinematic viscosity. The buoyancy term is used for representing the temperature and to simplify computation.

In this case, we non-dimensionalize the problem by using domain height $h = L_z = 1$ and the freefall time. Therefore, the Rayleigh number and Prandtl number can be determined by

$$Ra := \frac{1}{\nu \kappa}, \quad Pr := \frac{\nu}{\kappa} \quad (4)$$

Based on that, we can obtain the thermal diffusivity κ and kinematic viscosity ν .

$$\Rightarrow \kappa := (Ra Pr)^{-1/2}, \quad \nu := \left(\frac{Ra}{Pr} \right)^{-1/2} \quad (5)$$

```
[ ]: Ra = 3e6      # Rayleigh number
     Pr = 1.       # Prandtl number
     kappa = (Ra*Pr)**(-1./2.) # thermal diffusivity
     nu = (Ra/Pr)**(-1./2.)    # kinematic viscosity
```

Governing equations are modified to fit with the Dedalus framework:

$$\nabla \cdot \vec{u} = 0, \quad (6)$$

$$\partial_t \vec{u} - \nu \nabla^2 \vec{u} + \nabla p - B \vec{e}_z = -\vec{u} \cdot \nabla \vec{u}, \quad (7)$$

$$\partial_t B - \kappa \nabla^2 B = -\vec{u} \cdot \nabla B. \quad (8)$$

To begin with, we must create basis for problem, including the mesh system for present 2D problem and defining data type of output.

```
[ ]: dtype = np.float64 # = double
     dealias = 3/2

     # Bases
     coords = d3.CartesianCoordinates('x','z')
     dist = d3.Distributor(coords, dtype=dtype)
     xbasis = d3.RealFourier(coords['x'], size=Nx, bounds=(Xmin, Xmax),
                               ↪ dealias=dealias)
     zbasis = d3.ChebyshevT(coords['z'], size=Nz, bounds=(Zmin, Zmax),
                              ↪ dealias=dealias)
```

Now, we will instantiate fields which will appears within the problem, including velocity $\vec{u}$, buoyancy b and pressure p .

```
[ ]: # Fields
     p = dist.Field(name='p', bases=(xbasis,zbasis)) # create pressure field
     b = dist.Field(name='b', bases=(xbasis,zbasis)) # create buoyancy field
     u = dist.VectorField(coords, name='u', bases=(xbasis,zbasis)) # create velocity
                               ↪ field
```

First, we consider the incompressible flow condition's equation:

$$\nabla \cdot \vec{u} = 0 \quad (9)$$

When implementing this divergence equation, there is a few error if one variable is underdetermined. So, we will use Gauge condition in present Dedalus v3, read [here](#). Here, the divergence equation is expanded the system by adding a spatially-constant variable (τ_p) for solving pressure field. As a result, the equation is replaced by

$$\nabla \cdot \vec{u} + \tau_p = 0 \quad (10)$$

In Dedalus, we need to create a corresponding sub-field for τ_p

```
[ ]: # Substitutions
x, z = dist.local_grids(xbasis, zbasis)
ex, ez = coords.unit_vector_fields(dist)
lift_basis = zbasis.derivative_basis(1)
lift = lambda A: d3.Lift(A, lift_basis, -1)

# create constant sub-field for incompressible flow condition's equation
tau_p = dist.Field(name='tau_p')
# because this term is only a constant added to the equation, we don't need to
  ↳ instantiate it for bases system

# create constant sub-field for bouyancy term
tau_b1 = dist.Field(name='tau_b1', bases=xbasis)
tau_b2 = dist.Field(name='tau_b2', bases=xbasis)
grad_b = d3.grad(b) + ez*lift(tau_b1) # First-order reduction

tau_u1 = dist.VectorField(coords, name='tau_u1', bases=xbasis)
tau_u2 = dist.VectorField(coords, name='tau_u2', bases=xbasis)
grad_u = d3.grad(u) + ez*lift(tau_u1) # First-order reduction
```

When we have all the fields for solving, we must build the IVP problem and introduce fields to the solver. At this step, we don't have full field for solving all three governing equations, so we just introduce corresponding codes to perform this.

Build problem for the solver

```
[ ]: # Problem
problem = d3.IVP([p, tau_p, u, tau_u1, tau_u2, b, tau_b1, tau_b2],
  ↳ namespace=locals())
```

Adding the first equation

$$\text{tr}(\nabla \cdot \vec{u}) + \tau_p = 0 \quad (11)$$

```
[ ]: # equation 1
problem.add_equation("trace(grad_u) + tau_p = 0")
problem.add_equation("integ(p) = 0") # Pressure gauge
```

Adding the second equation

$$\partial_t \vec{u} - \nu \nabla^2 \vec{u} + \nabla p - B \vec{e}_z = -\vec{u} \cdot \nabla \vec{u} \quad (12)$$

```
[ ]: # equation 2
problem.add_equation("dt(u) - nu*div(grad_u) + grad(p) - b*ez + lift(tau_u2) = 0
↳ u@grad(u)")
```

Adding the third equation

$$\partial_t B - \kappa \nabla^2 B = -\vec{u} \cdot \nabla B \quad (13)$$

```
[ ]: # equation 3
problem.add_equation("dt(b) - kappa*div(grad_b) + lift(tau_b2) = - u@grad(b)")
```

Set up the boundary conditions

```
[ ]: # for buoyancy
problem.add_equation("b(z=Zmin) = 1")
problem.add_equation("b(z=Zmax) = 0")
# for velocity
problem.add_equation("u(z=Zmin) = 0")
problem.add_equation("u(z=Zmax) = 0")
```

Solver

```
[ ]: stop_sim_time = 50
timestepper = d3.RK222
# Solver
solver = problem.build_solver(timestepper)
solver.stop_sim_time = stop_sim_time
```

Set up initial conditions

```
[ ]: b.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
b['g'] *= z * (Zmax - z) # Damp noise at walls
b['g'] += Zmax - z # Add linear background
```

```
[ ]: # Analysis
import shutil, os
sim_name = 'Data_RBC_DNS'
if os.path.exists(sim_name):
    shutil.rmtree(sim_name) # remove the output directory and its previous
    ↳ contents

dataset = solver.evaluator.add_file_handler(sim_name, sim_dt=0.1,
    ↳ max_writes=100)
dataset.add_task(b, name='buoyancy')
dataset.add_task(u, name='velocity')
dataset.add_task(-d3.div(d3.skew(u)), name='vorticity')
```

```
[ ]: max_timestep = 0.125
# CFL
CFL = d3.CFL(solver, initial_dt=max_timestep, cadence=10, safety=0.5,
↳threshold=0.05,
        max_change=1.5, min_change=0.5, max_dt=max_timestep)
CFL.add_velocity(u)

[ ]: # Flow properties
flow = d3.GlobalFlowProperty(solver, cadence=10)
flow.add_property(np.sqrt(u@u)/nu, name='Re')

[ ]: from dedalus.extras.plot_tools import plot_bot_2d
# from dedalus.tools.array import *
# from src.plot_lib import *
figkw = {'figsize':(1,1), 'dpi':200}
# Main loop
print('Starting main loop')
while solver.proceed:
    timestep = CFL.compute_timestep()
    solver.step(timestep)
    if solver.iteration % 100 == 0:
        print('Completed iteration {}, time={:.3f}'.format(solver.iteration,
↳solver.sim_time))
        # print(u['g'][1])
        # u.layout.grid_space
        # plot_bot_2d(u, figkw=figkw, title="b['g']")
        # plt.savefig("bouyancy_t{:.3f}.png".format(solver.sim_time), dpi=200)
        # plt.close()
```